ÉCOLE SUPÉRIEURE D'INGÉNIEURS DE BEYROUTH
Université Saint-Joseph — Computer Engineering

End-of-Semester Project P4

Case-law research assistant

Faithfulness, citation verification and calibrated abstention

Team members	
Supervisor	
Repository URL	
Deployed URL	
Defence date	

Every data source named in this brief was fetched and verified on 21 September 2026.
Volumes and byte counts are from live responses on that date and will drift. Re-check before you rely on them.

1. What you are building

You are building a website that answers legal research questions using real court decisions — and, more importantly, that never invents one. Lawyers have been fined for filing documents that cited cases which did not exist, because a computer made them up and nobody checked. Your system checks. A visitor asks a question; your site answers using actual decisions, shows exactly which passage each statement came from, and confirms that every case it names really exists before showing the answer. If the decisions it found do not answer the question, it says so plainly instead of guessing. Saying "I don't know" at the right moment is the point of this project, not a weakness in it. The private area is a testing room, where the operator runs the system against known questions and watches for any invented reference.

The same thing, in the language of the course

Fabricated citations are the best-documented failure of retrieval-augmented generation in professional practice. Build the system that structurally cannot produce one.

Skill this project teaches	Faithfulness, citation verification and calibrated abstention
Subject area	Professional / legal research

What this project is really about

Each of the eight projects this semester teaches a different way these systems fail, and a different way to stop them failing. Yours is the one named above. The interface, the framework you choose and the amount of code you write are not what separates a good submission from a weak one here — section 7 is.

Requirements common to every project this semester

- Web-based, deployed publicly, with a live URL reachable at the defence.
- Two distinct surfaces — a user interface and an administrative interface — behind role-based authentication.
- A documented chunking strategy with a written justification. "We used 512 tokens" is not a justification.
- An evaluation set of at least fifty question and answer pairs, written by your team.
- Retrieval metrics reported before and after one deliberate improvement.
- Administrative analytics: query volume, latency, failure rate, and the questions the system could not answer.
- The licence of every ingested document recorded at ingestion time.

2. Your corpus

This corpus was verified before this brief was issued. You are not expected to go looking for data, and you should not substitute a different source without asking.

Primary source	CourtListener bulk data — https://storage.courtlistener.com/bulk-data/
Verified volume	opinions-2024-12-31.csv.bz2, 44,759,737,019 bytes, served without authentication. Also available: courts, dockets, opinion clusters, the citation map, parentheticals and judges.
Update cadence	Quarterly, on the last day of March, June, September and December.
Licence	Free of known copyright restrictions. Public Domain Mark. The cleanest licence in this pack.
Scope control	Teams must filter to a single court or a bounded date range. The full opinions file is not a semester-sized download and attempting it will consume the term.
Access note	The REST API now requires a token and returns HTTP 401 without one. The bulk files require nothing.

3. Acquiring the corpus

Every command below was executed successfully on 21 September 2026. They are starting points, not a complete pipeline.

1. List what is available before downloading. The bulk directory is browsable.

```
curl -sS "https://com-courtlistener-storage.s3-us-west-2.amazonaws.com/\
list.html?prefix=bulk-data/" | head -c 2000
```

2. Check the size before you start the transfer. The opinions file is 44.8 GB.

```
curl -sSI "https://storage.courtlistener.com/bulk-data/opinions-2024-12-31.csv.bz2" \
  | grep -i content-length
```

3. Take the courts and citation-map files first. They are small, and the citation map is what makes the verification stage possible.
4. Stream and filter rather than storing the whole file. Decompress on the fly, keep only your chosen court, discard the rest.
5. Fix your scope in writing in week one: which court, which date range, how many opinions. Put it in SPEC.md.

Week 1 gate — what you must show

> A filtered subset loaded and counted, the citation map loaded, and a working lookup that answers whether a given citation exists. The whole pipeline rests on that lookup.

This gate exists because corpus risk is the largest single risk in a retrieval project. Five projects in the first draft of this cohort were abandoned when their data sources failed verification. Yours survived that process — but you still have to get the data onto disk before you design around it.

4. Architecture requirements

- Retrieval and generation with span-level citation.
- A verification stage: every citation the model emits is checked against the citation map before the answer is returned to the user. Unverifiable citations are stripped and the answer is degraded accordingly.
- An explicit abstention path when retrieved support is insufficient.

5. User interface

- Research question answering with inline citations, each one clickable and each one verified before display.
- An explicit "this is not supported by the retrieved opinions" response, distinct from a low-confidence answer.
- Retrieved passages shown alongside the answer for inspection.

6. Administrative interface

The administrative interface is worth fifteen marks and is where most teams underinvest. It is not a table viewer. Ask yourself whether a person who did not build this system could operate it from these screens.

- Evaluation dashboard: golden question set, groundedness scoring, citation-existence checking.
- Regression run triggered automatically on every re-index.
- Failure browser for questions where the system abstained, so abstention can be tuned rather than guessed.

7. The hard part

Calibrated abstention. Knowing when not to answer is graded harder than answer quality.

What weeks 7 and 8 look like

Weeks 7 and 8 are spent on the verification stage and abstention calibration: build the citation check into the response path so an unverifiable citation cannot reach the user, then tune the abstention threshold against the test set.

What an excellent submission looks like

A reported false-citation rate of zero against an adversarial test set the team wrote specifically to break its own system.

Principal risk

Download size. A team that starts the 44 GB transfer in week 9 will not finish the project.

8. Evaluation requirements

Your evaluation set is a deliverable, not a convenience. It is written by your team, it is committed early, and its commit dates are checked. Twenty-five marks depend on this section.

- At least fifty question and answer pairs, of which at least fifteen have no supported answer in your subset. These test abstention, and a system that answers them anyway has failed.
- A false-citation rate, measured against the citation map. The target is zero.
- An adversarial set written by the team specifically to induce fabricated citations.
- Abstention precision and recall, reported separately. Abstaining on everything scores as badly as never abstaining.

The deliberate improvement

At some point you will change something in your retrieval pipeline that you believe makes it better. Measure the system before the change and after it, on the same test set, and report both numbers. If the change made things worse, report that. A measured regression that you understood is worth more marks than an unmeasured improvement you assert.

9. Twelve-week plan

Weeks marked as a gate are pass or repeat. A team that has not cleared a gate does not move on to the next block of work; it fixes what is behind it.

Week	Milestone	Gate
1	Corpus acquisition proof — see section 3.	Gate
2	Ingestion and chunking, with your written justification for the strategy chosen.	
3	Baseline retrieval working, and the fifty-question evaluation set written.	Gate
4	Generation layer with citations bound to retrieved spans.	
5	Authentication, roles, and the administrative interface skeleton.	
6	Mid-semester demonstration: end-to-end, deployed, unpolished but working.	Gate
7	The hard part — first implementation.	
8	The hard part — measured and corrected.	
9	Administrative tooling completed: analytics, triage, operational views.	
10	The deliberate improvement, with retrieval metrics before and after.	Gate
11	Deployment hardening, seeded data, robustness under a demonstration you did not rehearse.	
12	Defence.	Gate

10. Deliverables

Deliverable	Notes
Git repository	Shared with the lecturer from week one. The commit history is part of the assessment.
Live deployed URL	Reachable at the defence, with seeded data. Localhost is not a deployment.
SPEC.md	Committed and tagged before the first implementation commit.
EVALUATION.md	The test set, the method, and the measured results before and after your improvement.
AI_LOG.md	See section 11.
Technical report	Six pages maximum. Architecture, decisions, measurements, limitations.
Defence	Fifteen minutes plus questions. The live system, not slides of a system.

11. Building with AI assistance

This project is to be developed using AI coding tools. That is a requirement, not a permission. What is assessed is not whether you used the tool but whether you remained in command of what it produced.

11.1 Specification first

SPEC.md is committed and tagged before your first implementation commit. Your final system is assessed against it, or against your documented reasons for diverging from it. A specification written after the code is worth nothing and is usually obvious.

11.2 The AI log

AI_LOG.md records, for each significant delegation: what you asked, what came back, what was wrong with it, and how you caught it. A log in which nothing was ever wrong will be read as a log that was not kept.

11.3 Tests

You write your own tests. Generated tests that assert whatever the code already does score nothing, and are easy to recognise.

11.4 The rule that matters

Any code you cannot explain at the defence scores zero for that component. Commit authorship is expected to mean something.

11.5 The question every team is asked

> *Show me a place where the AI produced something wrong, and explain how you caught it.*

Prepare for this. A team that answers "it did not" has not looked, and will be marked as not having looked.

12. Assessment

Component	Weight	What is being assessed
Retrieval quality and evaluation rigour	25	Your test set, your metrics, the honesty of the measurement, and the before-and-after of your deliberate improvement.
The hard part (section 7)	20	The competency this project exists to teach. This is what separates your project from the other seven.
Administrative interface and operational tooling	15	Whether someone could actually run this system: triage, monitoring, correction, re-indexing.
AI-assisted development process	15	SPEC.md, commit history, AI_LOG.md, and your answer to the question in section 11.5.
User interface	10	Clarity, provenance display, and honest handling of low-confidence answers.
Deployment and reliability	10	A live system that survives a demonstration it did not rehearse.
Report and defence	5	Whether you can explain your own system.

Note the weighting. Retrieval quality and the hard part together account for forty-five marks. Interface work accounts for ten. A beautiful front end over untested retrieval has misread this brief.

12.1 Automatic deductions

- An evaluation set written in the final fortnight, evidenced by commit dates.
- Numeric or factual values in answers that cannot be traced to a retrieved source.
- An administrative interface that is a database table viewer with no operational function.
- Any code no team member can explain.

13. Technical traps in your sources

Each of these was encountered while verifying your corpus. Each will otherwise cost you days.

Source	Trap
CourtListener	The bulk files need no authentication. The REST API returns HTTP 401 without a token — do not conclude the data is unavailable.
CourtListener	44.8 GB compressed. A team that starts this download in week nine will not finish the project.
Method	Verifying that a citation exists is not the same as verifying it supports the claim. Say which one you implemented.

14. Verification and drift

Every endpoint in this brief was fetched on 21 September 2026 and returned what is described. Figures come from those responses.

Data sources change. Before you build around any number in section 2, re-run the relevant command in section 3 and check that it still holds. If it does not, tell your supervisor in week one rather than working around it.

If a source named here becomes unavailable during the semester, that is a supervisory matter, not a reason to silently substitute another corpus. Ask.

Nothing in this brief was asserted without being verified. The same standard applies to your report.